1. Create a Storage Account in Azure for Terraform State

```
askaniusz@askaniusz-Crosshair-16-HX-D14VFKG:~/Desktop/m07_sparksql_python_azure-master$ az group create --name [redacted] --location eastus
{
  "id": "/subscriptions[redacted]/resourceGroups/[redacted]",
  "location": "eastus",
  "managedBy": null,
  "name": "[redacted]",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
```

```
  },
  "keyPolicy": null,
  "kind": "StorageV2",
  "largeFileSharesState": null,
  "lastGeoFailoverTime": null,
  "location": "eastus",
  "minimumTlsVersion": "TLS1_0",
  "name": "[redacted]",
  "networkRuleSet": {
    "bypass": "AzureServices",
    "defaultAction": "Allow",
    "ipRules": [],
    "ipv6Rules": [],
    "resourceAccessRules": null,
    "virtualNetworkRules": []
  },
  "placement": null,
  "primaryEndpoints": {
    "blob": "https://[redacted]blob.core.windows.net/",
    "dfs": "https://s[redacted]dfs.core.windows.net/",
    "file": "https://s[redacted].file.core.windows.net/",
    "internetEndpoints": null,
    "ipv6Endpoints": null,
    "microsoftEndpoints": null,
    "queue": "https://[redacted]queue.core.windows.net/",
    "table": "https://s[redacted]s.table.core.windows.net/",
    "web": "https:/[redacted]z13.web.core.windows.net/"
  },
```

```
askaniusz@askaniusz-Crosshair-16-HX-D14VFKG:~/Desktop/m07_sparksql_python_azure-master$ az storage container
 create --name tfstate --account-name [redacted]

There are no credentials provided in your command and environment, we will query for account key for your st
orage account.
It is recommended to provide --connection-string, --account-key or --sas-token in your command as credential
s.

You also can add `--auth-mode login` in your command to use Azure Active Directory (Azure AD) for authorizat
ion if your login account is assigned required RBAC roles.
For more information about RBAC roles in storage, visit https://learn.microsoft.com/azure/storage/common/sto
rage-auth-aad-rbac-cli.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command
. Please use --help to get more information about environment variable usage.
{
  "created": true
}
```

3. Update Terraform Configuration

main.tf

terraform > main.tf

```
# Setup azurerm as a state backend
terraform {
  backend "azurerm" {
    resource_group_name  = [redacted]"
    storage_account_name = "[redacted]" # Provide Storage Account name, where Terraform Remote state is stored
    container_name       = "[redacted]"
    key                  = [redacted]
  }
}

# Configure the Microsoft Azure Provider
provider "azurerm" {
  features {}
  subscription_id = [redacted]
}
```

4. Deploy Infrastructure with Terraform

```
askaniusz@askaniusz-Crosshair-16-HX-D14VFKG:~/Desktop/m07_sparksql_python_azure-master/terraform$ terrafo
output resource_group_name
"rg-d[redacted]pv"
askaniusz@askaniusz-Crosshair-16-HX-D14VFKG:~/Desktop/m07_sparksql_python_azure-master/terraform$
```

Name ↑	Subscription	Location
databricks-rg-rg-de	Azure subscription 1	West Europe
Network\	Azure subscription 1	West Europe
rg-dat	Azure subscription 1	East US
rg-de	Azure subscription 1	West Europe

Home › Resource Manager | Resource groups › Containers

data
Container

Successfully uploaded blob(s)
Successfully uploaded 973 blob(s).

Search

- Overview
- Diagnose and solve problems
- Access Control (IAM)
- Settings
  - Shared access tokens
  - Manage ACL
  - Access policy
  - Properties
  - Metadata

Add Directory | Upload | Refresh | Delete | Copy | Paste | Rename | Acquire lease

data

Authentication method: Access key (Switch to Microsoft Entra user account)

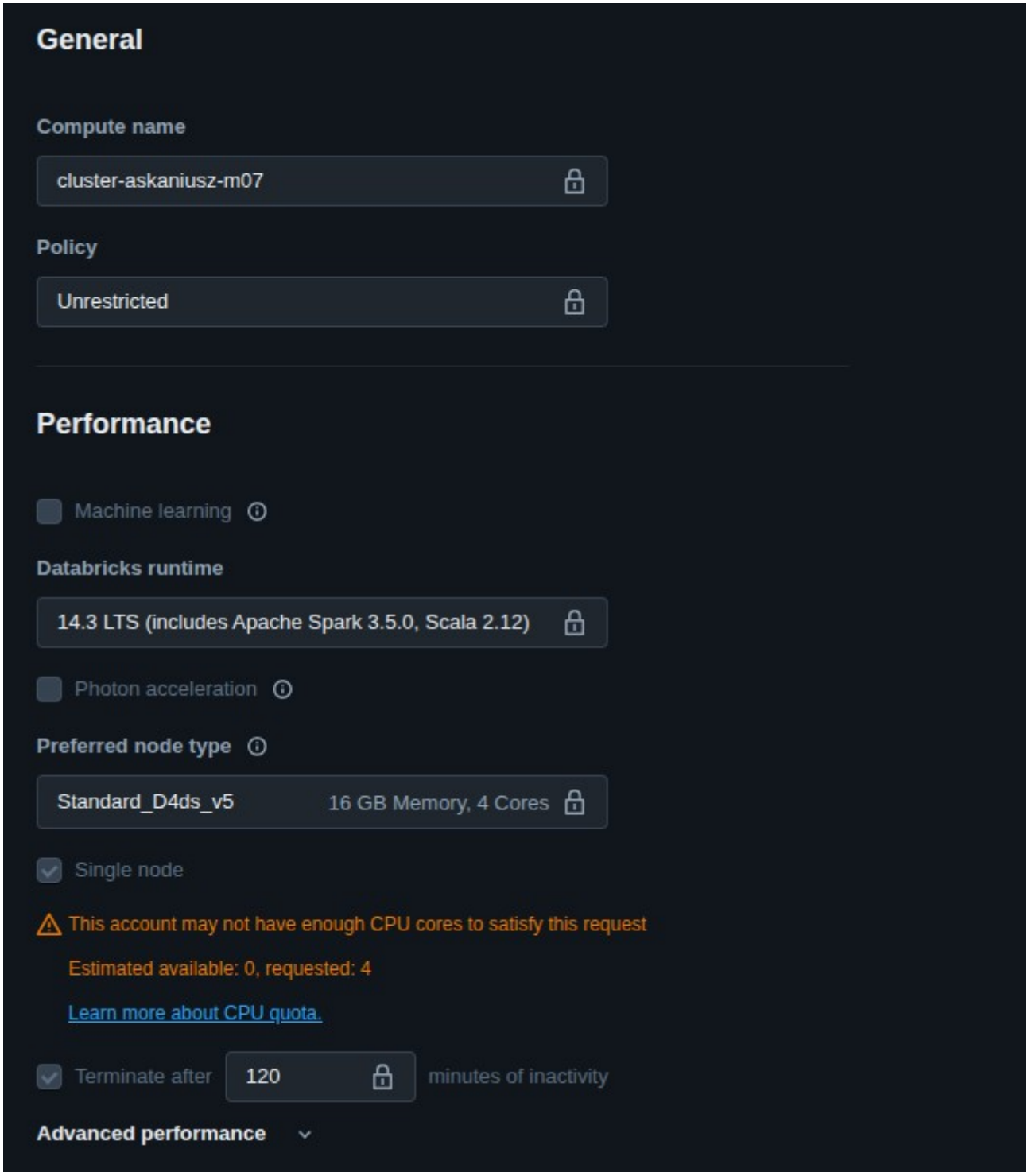
Search blobs by prefix (case-sensitive) | Only show active objects

Showing all 2 items

Name	Last modified	Access tier	Blob type	Size	Lease state
expedia	2/21/2026, 10:40:17 AM				
hotel-weat...	2/21/2026, 10:44:11 AM				

6. Launch notebooks on Databricks cluster

Create cluster



Create Databricks Notebook → 01_create_metadata

Create three schemas: bronze, silver and gold.

01_create_metadata

Python | Tabs: ON

11:58 AM (3s) 1

```python
storage_account = "[redacted]"
container = "data"
base_path = f"abfss://{container}@{storage_account}.dfs.core.windows.net"

# 1. Create the physical directories on ADLS
for layer in ["bronze", "silver", "gold"]:
    dbutils.fs.mkdirs(f"{base_path}/{layer}")

# 2. Create the Metastore Schemas
spark.sql(f"CREATE SCHEMA IF NOT EXISTS bronze LOCATION '{base_path}/bronze'")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS silver LOCATION '{base_path}/silver'")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS gold LOCATION '{base_path}/gold'")
```

Table

	database_description_item	database_description_value
1	Catalog Name	spark_catalog
2	Namespace Name	bronze
3	Comment	
4	Location	abfss://data@std[redacted]pv.dfs.core.v
5	Owner	root
6	Properties	

6 rows | 2.51s runtime Refreshed 9 minutes ago

Encrypt Personally Identifiable Information (PII) fields to protect sensitive data.

02_load_bronze_data | pii_utils

Python | Tabs: ON | Last edit was 3 minutes ago

3 minutes ago (<1s) 1 Python

```python
from pyspark.sql.functions import aes_encrypt, aes_decrypt, lit, col

class PIIProcessor:
    def __init__(self, encryption_key):
        self.encryption_key = encryption_key

    def encrypt_columns(self, df, columns_to_encrypt):
        for column in columns_to_encrypt:
            df = df.withColumn(column, aes_encrypt(col(column).cast("string").cast("binary"), lit
            (self.encryption_key)))
        return df

    def decrypt_columns(self, df, columns_to_decrypt):
        for column in columns_to_decrypt:
            # Deszyfrujemy i rzutujemy na string, żeby wynik był czytelny
            df = df.withColumn(column, aes_decrypt(col(column), lit(self.encryption_key)).cast
            ("string"))
        return df
```

[Shift+Enter] to run and move to next cell

Load bronze data

2 minutes ago (<1s) 1 Python

```
%run ./pii_utils
```

1 minute ago (40s) 2 Python

```python
storage_account = "[redacted]"
container = "data"
base_url = f"abfss://{container}@{storage_account}.dfs.core.windows.net"
MY_KEY = "[redacted]" 16 lengths

processor = PIIProcessor(MY_KEY)

# Read Expedia
df_expedia = spark.read.format("avro").load(f"{base_url}/expedia")

# Encryption id and user_location_city
expedia_pii = ["user_id", "user_location_city"]
df_expedia_secure = processor.encrypt_columns(df_expedia, expedia_pii)

# Write bronze data expedia
df_expedia_secure.write.format("delta").mode("overwrite").saveAsTable("bronze.expedia")

# Read hotel-weather
df_hw = spark.read.format("parquet").load(f"{base_url}/hotel-weather")
# Adress encryption
hw_pii = ["address"]
df_hw_secure = processor.encrypt_columns(df_hw, hw_pii)

# Write bronze data hotel-weather
df_hw_secure.write.format("delta").mode("overwrite").saveAsTable("bronze.hotel_weather")
```

(13) Spark Jobs

```
df_expedia: pyspark.sql.dataframe.DataFrame = [id: long, date_time: string ... 18 more fields]
df_expedia_secure: pyspark.sql.dataframe.DataFrame = [id: long, date_time: string ... 18 more fields]
df_hw: pyspark.sql.dataframe.DataFrame = [address: string, avg_tmpr_c: double ... 12 more fields]
```

Home > ' Containers

data

Container

Search

- Overview
- Diagnose and solve problems
- Access Control (IAM)
- Settings
  - Shared access tokens
  - Manage ACL
  - Access policy
  - Properties
  - Metadata

Add Directory | Upload | Refresh | Delete | Copy | Paste | Rename | Acquire lease | Break lea

data > bronze

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Showing all 2 items

Name
[..]
expedia
hotel_weather

Bronze to Silver

12:33 PM (<1s) 1

```
%run ./pii_utils
```

12:37 PM (<1s) 2

```
from pyspark.sql.functions import col, trim, to_timestamp, nullif, when

# 2. Key initialization
MY_KEY = [redacted]
processor = PIIProcessor(MY_KEY)

# 3. Read bronze table
df_expedia = spark.table("bronze.expedia")
df_hotel_weather = spark.table("bronze.hotel_weather")
```

```
df_expedia: pyspark.sql.dataframe.DataFrame = [id: long, date_time: string ... 18 more fields]
df_hotel_weather: pyspark.sql.dataframe.DataFrame = [address: binary, avg_tmpr_c: double ... 12 more fields]
```

1 minute ago (50s) 3 Python

```python
# Decryption expedia
expedia_pii = ["user_id", "user_location_city"]
df_ex_decrypted = processor.decrypt_columns(df_expedia, expedia_pii)

# Cleaning
df_ex_cleaned = df_ex_decrypted \
    .withColumn("date_time", to_timestamp(col("date_time"))) \
    .withColumn("srch_ci", to_timestamp(col("srch_ci"))) \
    .withColumn("srch_co", to_timestamp(col("srch_co"))) \
    .filter(col("id").isNotNull()) \
    .dropDuplicates()

# Again encryption
df_ex_final = processor.encrypt_columns(df_ex_cleaned, expedia_pii)

# Write to silver
df_ex_final.write.format("delta").mode("overwrite").saveAsTable("silver.expedia_processed")
```

(8) Spark Jobs

```
df_ex_cleaned:  pyspark.sql.dataframe.DataFrame = [id: long, date_time: timestamp ... 18 more fields]
df_ex_decrypted:  pyspark.sql.dataframe.DataFrame = [id: long, date_time: string ... 18 more fields]
df_ex_final:  pyspark.sql.dataframe.DataFrame = [id: long, date_time: timestamp ... 18 more fields]
```

Just now (6s) 4 Python

```python
# Decryption hotel_weather
hw_pii = ["address"]
df_hw_decrypted = processor.decrypt_columns(df_hotel_weather, hw_pii)

# Cleaning
df_hw_cleaned = df_hw_decrypted \
    .withColumn("address", trim(col("address"))) \
    .withColumn("name", trim(col("name"))) \
    .withColumn("wthr_date", to_timestamp(col("wthr_date"))) \
    .dropna(subset=["id", "wthr_date"]) \
    .distinct()

# Encryption
df_hw_final = processor.encrypt_columns(df_hw_cleaned, hw_pii)

# Write to silver
df_hw_final.write.format("delta").mode("overwrite").saveAsTable("silver.hotel_weather_processed")
```

(8) Spark Jobs

```
df_hw_cleaned:  pyspark.sql.dataframe.DataFrame = [address: string, avg_tmpr_c: double ... 12 more fields]
df_hw_decrypted:  pyspark.sql.dataframe.DataFrame = [address: string, avg_tmpr_c: double ... 12 more fields]
df_hw_final:  pyspark.sql.dataframe.DataFrame = [address: binary, avg_tmpr_c: double ... 12 more fields]
```

Home › Containers

data
Container

Search

- Overview
- Diagnose and solve problems
- Access Control (IAM)
- Settings
  - Shared access tokens
  - Manage ACL
  - Access policy
  - Properties
  - Metadata

Add Directory | Upload | Refresh | Delete | Copy | Paste | Rename | Acquire lease | ...

data > silver

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 2 items

Name	Last modified	Access tier	Blob type	Siz
[..]				
expedia_processed	2/21/2026, 12:41:54 PM			
hotel_weather_processed	2/21/2026, 12:43:26 PM			

Silver to Gold

04_silver_to_gold

Python | Tabs: ON | Last edit was now

Run cell (Ctrl + Enter)

12:47 PM (<1s) 1 Python

```
%run ./pii_utils
```

12:51 PM (<1s) 2

```
from pyspark.sql.functions import col, abs, count, max, min, last, first, datediff, month, avg

# Initizalization
MY_KEY = [redacted]
processor = PIIProcessor(MY_KEY)

# Read silver table
df_expedia = spark.table("silver.expedia_processed")
df_hw = spark.table("silver.hotel_weather_processed")

# Decryption
df_expedia_dec = processor.decrypt_columns(df_expedia, ["user_id", "user_location_city"])
df_hw_dec = processor.decrypt_columns(df_hw, ["address"])

# Crate TempView
df_expedia_dec.createOrReplaceTempView("expedia")
df_hw_dec.createOrReplaceTempView("weather")
```

```
df_expedia: pyspark.sql.dataframe.DataFrame = [id: long, date_time: timestamp ... 18 more fields]
df_expedia_dec: pyspark.sql.dataframe.DataFrame = [id: long, date_time: timestamp ... 18 more fields]
df_hw: pyspark.sql.dataframe.DataFrame = [address: binary, avg_tmpr_c: double ... 12 more fields]
df_hw_dec: pyspark.sql.dataframe.DataFrame = [address: string, avg_tmpr_c: double ... 12 more fields]
```

4 minutes ago (4s) 3

```
# Calculate max and min temp for hotel and month
top_10_temp_diff = spark.sql("""
    SELECT
        id as hotel_id,
        name,
        month,
        (MAX(avg_tmpr_c) - MIN(avg_tmpr_c)) as temp_diff
    FROM weather
    GROUP BY id, name, month
    ORDER BY ABS(temp_diff) DESC
    LIMIT 10
""")

top_10_temp_diff.write.format("delta").mode("overwrite").saveAsTable("gold.
top_10_temp_diff_monthly")
```

(8) Spark Jobs

```
top_10_temp_diff: pyspark.sql.dataframe.DataFrame = [hotel_id: string, name: string ... 2 more fields]
```

4 minutes ago (5s) 4 Python

```python
# Top 10 busiest hotels monthly
top_10_busiest = spark.sql("""
    SELECT
        hotel_id,
        month(to_date(srch_ci)) as month,
        COUNT(*) as visit_count
    FROM expedia
    GROUP BY hotel_id, month
    ORDER BY visit_count DESC
    LIMIT 10
""")

top_10_busiest.write.format("delta").mode("overwrite").saveAsTable("gold.
top_10_busiest_hotels_monthly")
```

(8) Spark Jobs

top_10_busiest: pyspark.sql.dataframe.DataFrame = [hotel_id: long, month: integer ... 1 more field]

1 minute ago (5s) 5 Python

```python
# We use a CTE (WITH) to first calculate the statistics across windows
# and then extract the unique rows for each reservation
weather_trend = spark.sql("""
    WITH stay_stats AS (
        SELECT
            e.hotel_id,
            e.srch_ci as check_in,
            e.srch_co as check_out,
            DATEDIFF(e.srch_co, e.srch_ci) as stay_duration,
            w.avg_tmpr_c,
            w.wthr_date,
            -- Średnia temperatura w oknie całej rezerwacji
            AVG(w.avg_tmpr_c) OVER (PARTITION BY e.hotel_id, e.srch_ci, e.srch_co) as
            avg_temp_during_stay,
            -- Pierwsza i ostatnia temperatura w pobycie
            FIRST_VALUE(w.avg_tmpr_c) OVER (PARTITION BY e.hotel_id, e.srch_ci, e.srch_co ORDER
            BY w.wthr_date) as first_day_temp,
            LAST_VALUE(w.avg_tmpr_c) OVER (PARTITION BY e.hotel_id, e.srch_ci, e.srch_co ORDER BY
            w.wthr_date
                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as last_day_temp
        FROM expedia e
        JOIN weather w ON e.hotel_id = w.id AND w.wthr_date BETWEEN e.srch_ci AND e.srch_co
        WHERE DATEDIFF(e.srch_co, e.srch_ci) > 7
    )
    SELECT DISTINCT
        hotel_id,
        check_in,
        check_out,
        stay_duration,
        avg_temp_during_stay,
        (last_day_temp - first_day_temp) as temp_trend
    FROM stay_stats
""")

weather_trend.write.format("delta").mode("overwrite").saveAsTable("gold.
weather_trend_extended_stay")
```

(8) Spark Jobs

weather_trend: pyspark.sql.dataframe.DataFrame = [hotel_id: long, check_in: timestamp ... 4 more fields]

1

CHECK

Just now (<1s) 6 Python

```
print("TOP 10 HOTELS - MONTHLY TEMP DIFFERENCE:")
display(spark.table("gold.top_10_temp_diff_monthly"))
```

(1) Spark Jobs

```
TOP 10 HOTELS - MONTHLY TEMP DIFFERENCE:
```

Table

	hotel_id	name	month	temp_diff
1	1571958030336	316 N Main St	9	19.1
2	1571958030336	316 N Main St	10	18.5
3	146028888072	2799 W Airport Way	9	18.5
4	77309411328	Po Box 1327	10	18.2
5	695784701957	2011 Rodgers Dr	10	16.3
6	369367187456	555 E Lafayette Bl...	10	16.2000000000000...
7	146028888068	501 National St	9	15.5000000000000...
8	25769803779	663 E Cloverland Dr	9	15.1
9	1520418422787	2 Little Beaver Rd	10	14.8999999999999...
10	77309411335	1400 Historic Dr	10	14.8999999999999...

10 rows | 0.28s runtime Refreshed now

Just now (<1s) 7 Python

```python
print("\nTOP 10 BUSIEST HOTELS MONTHLY:")
display(spark.table("gold.top_10_busiest_hotels_monthly"))
```

(1) Spark Jobs

TOP 10 BUSIEST HOTELS MONTHLY:

Table

	hotel_id	month	visit_count
1	455266533377	9	416
2	2491081031684	8	408
3	2018634629125	9	407
4	1864015806473	10	405
5	42949672963	8	403
6	1829656068103	9	397
7	1881195675654	9	397
8	2448131358721	9	396
9	1958505086979	10	396
10	936302870530	10	395

10 rows | 0.20s runtime Refreshed now

Just now (<1s) 8

```python
print("\nWEATHER TREND FOR EXTENDED STAYS:")
display(spark.table("gold.weather_trend_extended_stay"))
```

(1) Spark Jobs

WEATHER TREND FOR EXTENDED STAYS:

Table

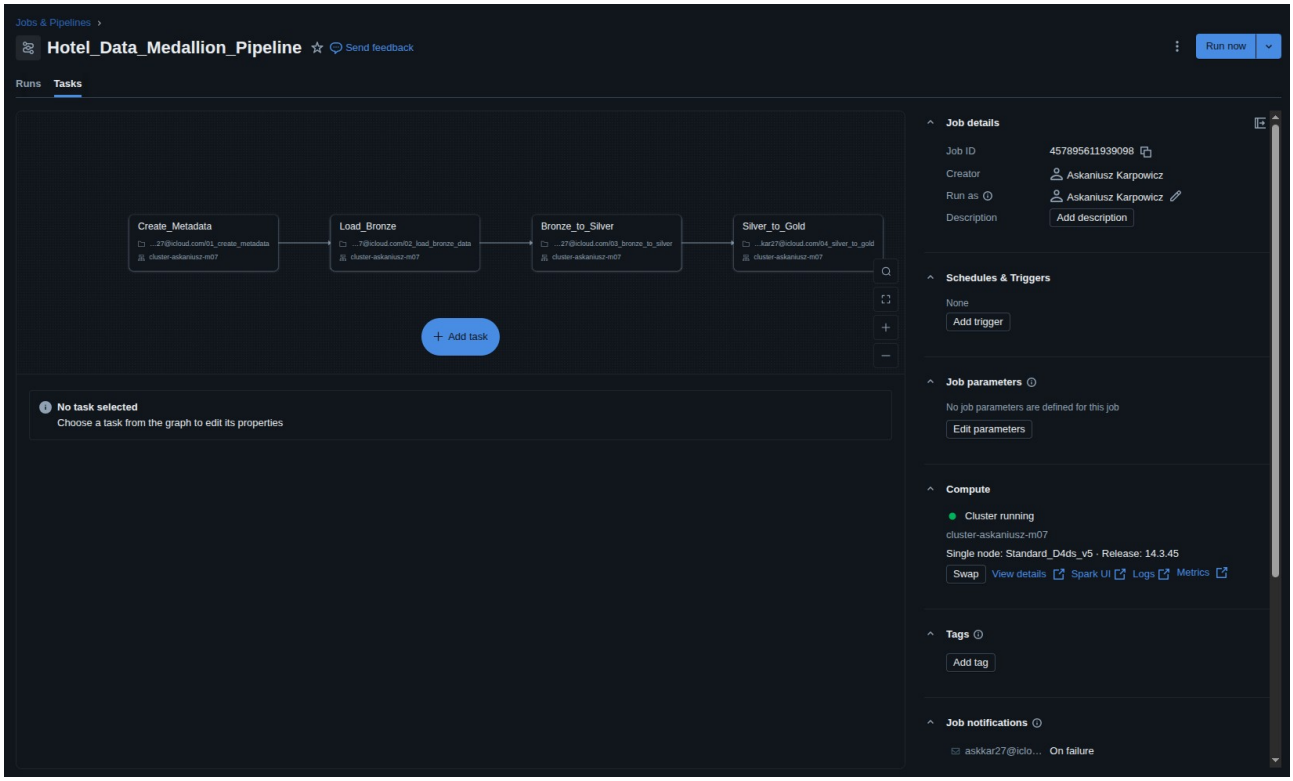
	hotel_id	check_in	check_out	stay_duration	avg_temp_during_stay	temp_trend
15	1	2016-10-22T00:00:00.000+00...	2016-10-21T00:00:00.000+00...	8	11.8	0
16	1	2016-10-25T00:00:00.000+00...	2016-11-08T00:00:00.000+00:...	14	11.8	0
17	1	2016-10-27T00:00:00.000+00...	2016-11-04T00:00:00.000+00:...	8	11.8	0
18	1	2017-08-11T00:00:00.000+00:...	2017-08-26T00:00:00.000+00...	15	21.1	-3.5999999999999998
19	1	2017-08-18T00:00:00.000+00...	2017-09-06T00:00:00.000+00...	19	19.9	1.1999999999999993
20	1	2017-08-21T00:00:00.000+00...	2017-09-02T00:00:00.000+00...	12	19.3	0
21	1	2017-08-22T00:00:00.000+00...	2017-08-31T00:00:00.000+00...	9	19.3	0
22	1	2017-08-24T00:00:00.000+00...	2017-09-05T00:00:00.000+00...	12	19.9	1.1999999999999993
23	1	2017-08-25T00:00:00.000+00...	2017-09-03T00:00:00.000+00...	9	19.3	0
24	1	2017-08-28T00:00:00.000+00...	2017-09-06T00:00:00.000+00...	9	20.5	0
25	1	2017-09-03T00:00:00.000+00...	2017-09-11T00:00:00.000+00:...	8	20.5	0
26	1	2017-09-13T00:00:00.000+00...	2017-09-21T00:00:00.000+00...	8	25.5	0
27	1	2017-09-21T00:00:00.000+00...	2017-09-29T00:00:00.000+00...	8	25.8	0
28	1	2017-09-23T00:00:00.000+00...	2017-10-23T00:00:00.000+00...	30	25.8	0
29	1	2017-09-24T00:00:00.000+00...	2017-10-02T00:00:00.000+00...	8	25.8	0

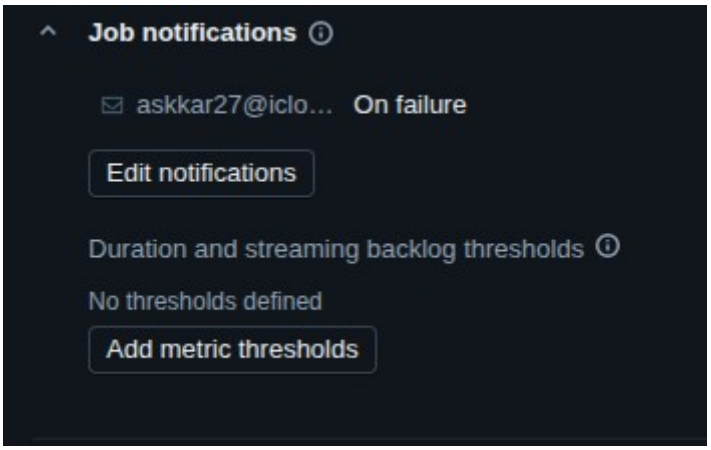
10,000+ rows | Truncated data | 0.43s runtime

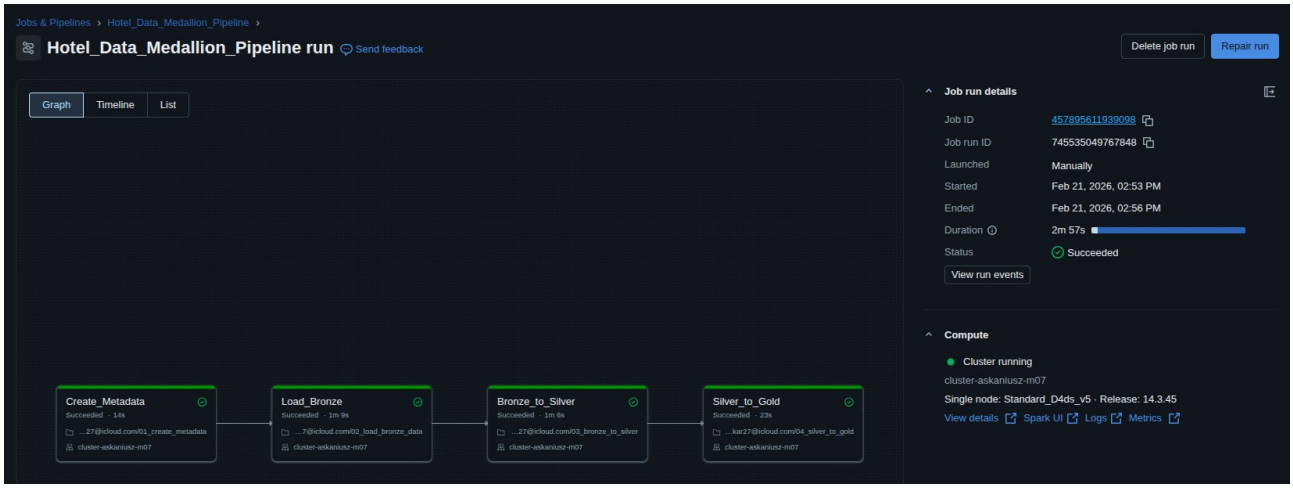
+ Code + Text Assistant

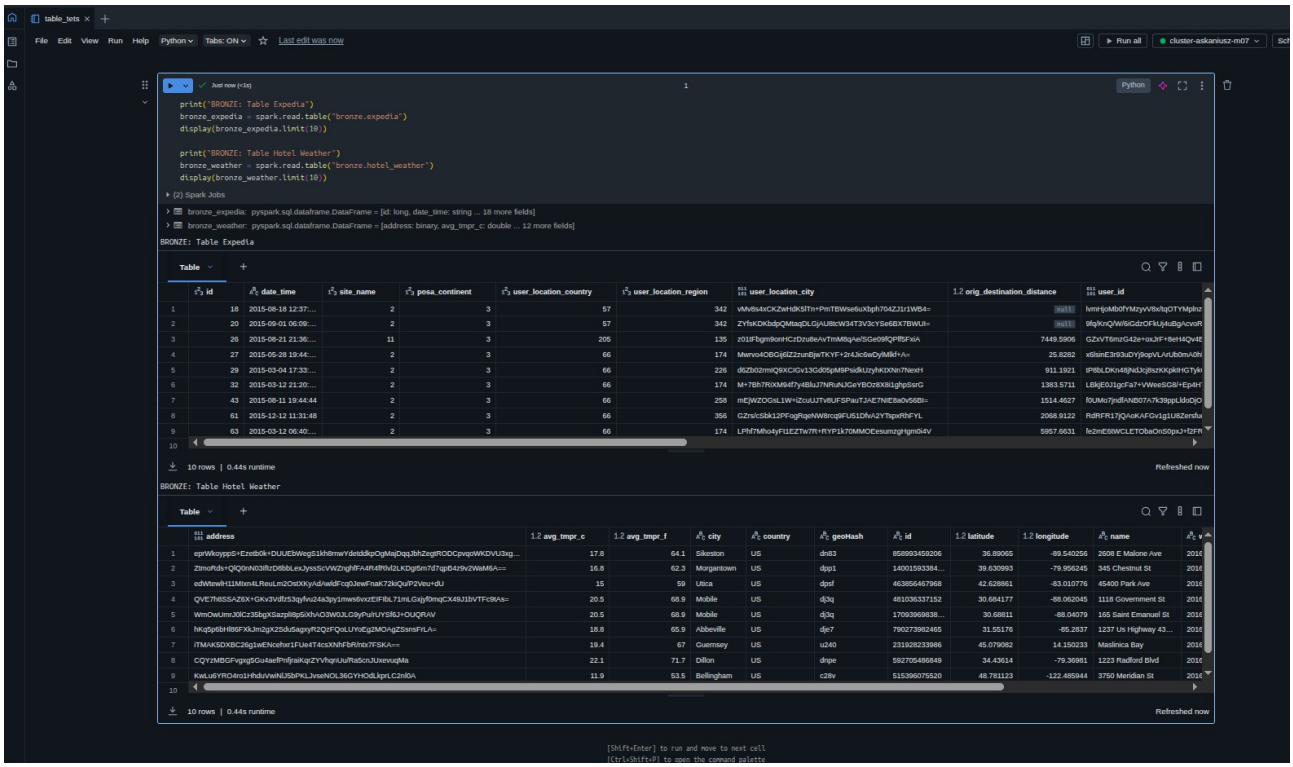
[Shift+Enter] to run and move to next cell

Orcjestrate the Jobs via Databricks Workflow


Jobs & Pipelines
Hotel_Data_Medallion_Pipeline
Send feedback
Run now
Runs
Tasks
Create_Metadata
Load_Bronze
Bronze_to_Silver
Silver_to_Gold
Add task
No task selected
Choose a task from the graph to edit its properties
Job details
Job ID
457895611939098
Creator
Askaniusz Karpowicz
Run as
Askaniusz Karpowicz
Description
Add description
Schedules & Triggers
None
Add trigger
Job parameters
No job parameters are defined for this job
Edit parameters
Compute
Cluster running
cluster-askaniusz-m07
Single node: Standard_D4ds_v5 · Release: 14.3.45
Swap
View details
Spark UI
Logs
Metrics
Tags
Add tag
Job notifications
askkar27@iclo...
On failure


Job notifications
askkar27@iclo...
On failure
Edit notifications
Duration and streaming backlog thresholds
No thresholds defined
Add metric thresholds





Check data after Workflow

+ Code + Text Assistant

Just now (1s) 2 Python

```python
print("Silver")
silver_data = spark.read.table("silver.expedia_processed")
display(silver_data.limit(10))

silver_data = spark.read.table("silver.hotel_weather_processed")
display(silver_data.limit(10))
```

(2) Spark Jobs

silver_data: pyspark.sql.dataframe.DataFrame = [address: binary, avg_tmpr_c: double ... 12 more fields]

```
Silver
```

Table

	id	date_time	site_name	posa_continent	user_location_country	user_location_region	user_location_city	orig_destination_distance	user_id
1	774	2015-07-07T19:12:47.000+00:...	11	3	205	411	CwbrqJnvfiRZGADuZ7WHTP9kcDXNqLg2CiMcc0xJ2oi/	143.2126	mq0xBm54u1+LN/4W+Dc
2	1309	2015-05-13T10:57:13.000+00:...	37	1	69	879	FTQ8cM9GkhLuDjJZ34HREjx50nt1zXSkQfimwAUKPhJP	null	rFa/+koVV0/qThuwjq2yQc
3	2423	2015-04-14T04:45:00.000+00:...	2	3	66	314	Va84NdETZ4I8osAqLp0DxxUCn1dqAdH8G5WTBgcpbFfu	51.5483	hvOD0DFezJKtxhLWlvlyfn
4	2855	2015-05-12T15:49:37.000+00:...	2	3	66	351	BCMeGGBulp6QQ5kac9iALZEYko1ilRhzp+piuxhW0qTr	null	DKpNEPqUBMK2U1Gk+L
5	3895	2015-10-01T17:10:18.000+00:...	2	3	239	359	6DEVliRxSpbodWzxUnp+6j90E+X81CaoVt3/3MQojY=	null	DKRpVGiRTXc+FqOoscp
6	7614	2015-10-11T00:46:26.000+00:...	2	3	66	315	rF5gI8WFGRFJW9STfidc6foZApzG2y6vvPLiBV/htuTRj	113.8083	5DkuBgrk33DIRW+WK13
7	21165	2015-05-30T16:25:58.000+00:...	2	3	66	220	F9FyNVDBsphTlisDVMOHJzeRNaz0R1GequAhgZSaNHea	791.3917	HeGNlR9VyrVALRMJPBJ
8	24779	2015-04-18T11:41:46.000+00:...	2	3	66	174	hEDzKNvPTSJlUdU+JENidazg+WetTydebA4W7zucTamv	28.5303	4QJ3f44T9BFHjPAF4vYtV
9	25349	2015-05-12T17:33:59.000+00:...	2	3	66	174	X+la+hKPRktjd+ZbCNfRSM0M3FWGD/5X9CjOasmWUb9...	678.6969	qpboJ9YtfjM9wwmzl2izwt
10									

10 rows | 1.14s runtime Refreshed now

Table

	address	avg_tmpr_c	avg_tmpr_f	city	country	geoHash	id	latitude	longitude
1	O+EikG4TDAwCOb2VK4OXnbrOPe/3lTmfUfCs9aEQOWPSctszSts8/GsYXx/w	17.9	64.2	Paris	FR	u09w	27487790694...	48.8693409	2.3310263
2	LUEkPjhLHKAD2mcXF++wJ7R7PwxJEWSvr/CC6eiWHLr+HBVAPQFtg==	16	60.8	London	GB	gcpu	26027501813...	51.4892447	-0.1803196
3	GxeKMEdzWPJ9bdtHibGgr/QWWV7tpxfb5n9IQyjDyrkCkNn/zRF8E5UgPeNMqHFBrKIlhY+O8	17.6	63.7	Paris	FR	u09t	28346764153...	48.8519208	2.338878
4	UpIl/70+PFi4sZ8cGnNoTeuMWHVExgXGMHduFdK5Gti3WZCz47Z3Qlh/IAPeUymsAaErY9zw	17.6	63.7	Paris	FR	u09t	20100446945...	48.8453785	2.3753036
5	jlIz/PA0G4Ants2WNj9hlxQvTmaz3h6ofVIuJkfnEhTUi/c+xkxvDOm9KQ=	17.6	63.7	Paris	FR	u09t	18640158064...	48.8663208	2.3450399
6	AjSqtlE42lB7oFQniTOWF1DJBpft+V9ZQMVIAHjXBtNxEe+IZwx6rcNo=	22.4	72.3	Milan	IT	u0nd	18468359372...	45.4667717	9.1871001
7	fBzGZC2HY2Mr+aQntIrHVt3XbCSHuH6l9bBU2bL91UMXZR6d8imQ	15	59	San Franci...	US	9q8z	13915694039...	37.799801	-122.436031
8	MHqnfNYRH4oOn5kEboT7WZZwKwYUQQ0b+P69RSKjZWEKegUdMML4jLZedfwK1A==	22.5	72.5	Boston	US	drt2	816043786245	42.346133	-71.071032
9	6tIF3g/MGCoQeuZmwxHgNSh7kgyrAGYDmgrYN1NiLdBagB3DO5b708BAZoR3qZYRlBQYM8XLG2Ect+WWybUyLoszq5Q63Xdvug...	26.6	79.9	Mont Belvieu	US	d5dn	747324309504	20.90572	-86.84786
10									

10 rows | 1.14s runtime Refreshed now

[Shift+Enter] to run and move to next cell

Just now (<1s) 3

```python
print("--- GOLD: Top 10 Busiest Hotels Monthly ---")
display(spark.read.table("gold.top_10_bustest_hotels_monthly").limit(10))
```

(1) Spark Jobs

```
--- GOLD: Top 10 Busiest Hotels Monthly ---
```

Table

	hotel_id	month	visit_count
1	455266533377	9	416
2	2491081031684	8	408
3	2018634629125	9	407
4	1864015806473	10	405
5	42949672963	8	403
6	1829656068103	9	397
7	1881195675654	9	397
8	2448131358721	9	396
9	1958505086979	10	396
10	936302870530	10	395

10 rows | 0.19s runtime Refreshed now

+ Code + Text Assistant

Just now (<1s) 4

```python
print("--- GOLD: Top 10 Temp Diff Monthly ---")
display(spark.read.table("gold.top_10_temp_diff_monthly").limit(10))
```

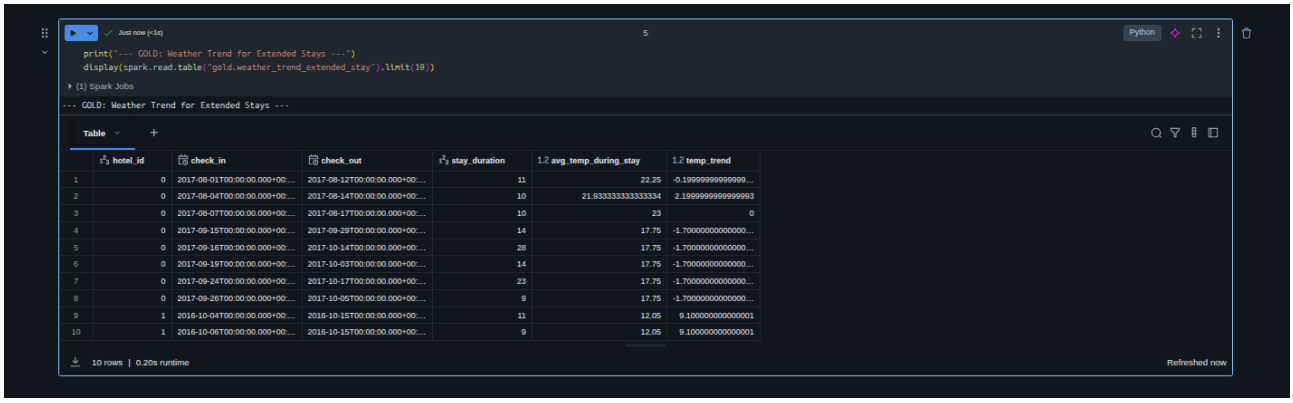
(1) Spark Jobs

```
--- GOLD: Top 10 Temp Diff Monthly ---
```

Table

	hotel_id	name	month	temp_diff
1	1571958030336	316 N Main St	9	19.1
2	1571958030336	316 N Main St	10	18.5
3	146028888072	2799 W Airport Way	9	18.5
4	77309411328	Po Box 1327	10	18.2
5	695784701957	2011 Rodgers Dr	10	16.3
6	369367187456	555 E Lafayette B...	10	16.2000000000000...
7	146028888068	501 National St	9	15.5000000000000...
8	25769803779	663 E Cloverland Dr	9	15.1
9	1520418422787	2 Little Beaver Rd	10	14.8999999999999...
10	77309411335	1400 Historic Dr	10	14.8999999999999...

10 rows | 0.23s runtime Refreshed now



```
print("--- GOLD: Weather Trend for Extended Stays ---")
display(spark.read.table("gold.weather_trend_extended_stay").limit(10))
```

(1) Spark Jobs

```
--- GOLD: Weather Trend for Extended Stays ---
```

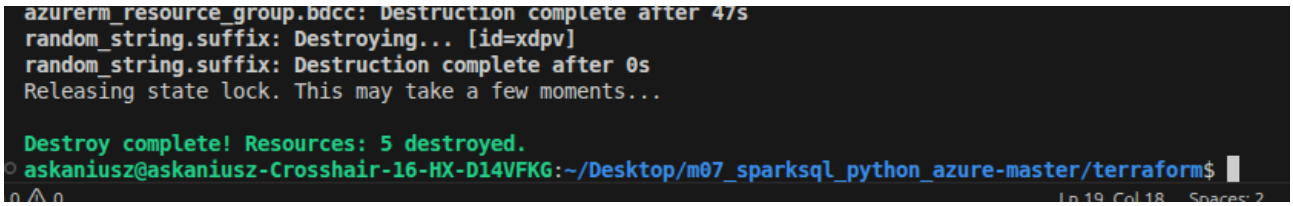
Table

	hotel_id	check_in	check_out	stay_duration	avg_temp_during_stay	temp_trend
1	0	2017-08-01T00:00:00.000+00:...	2017-08-12T00:00:00.000+00:...	11	22.25	-0.19999999999999...
2	0	2017-08-04T00:00:00.000+00:...	2017-08-14T00:00:00.000+00:...	10	21.933333333333334	2.1999999999999993
3	0	2017-08-07T00:00:00.000+00:...	2017-08-17T00:00:00.000+00:...	10	23	0
4	0	2017-09-15T00:00:00.000+00:...	2017-09-29T00:00:00.000+00:...	14	17.75	-1.70000000000000...
5	0	2017-09-16T00:00:00.000+00:...	2017-10-14T00:00:00.000+00:...	28	17.75	-1.70000000000000...
6	0	2017-09-19T00:00:00.000+00:...	2017-10-03T00:00:00.000+00:...	14	17.75	-1.70000000000000...
7	0	2017-09-24T00:00:00.000+00:...	2017-10-17T00:00:00.000+00:...	23	17.75	-1.70000000000000...
8	0	2017-09-26T00:00:00.000+00:...	2017-10-05T00:00:00.000+00:...	9	17.75	-1.70000000000000...
9	1	2016-10-04T00:00:00.000+00:...	2016-10-15T00:00:00.000+00:...	11	12.05	9.100000000000001
10	1	2016-10-06T00:00:00.000+00:...	2016-10-15T00:00:00.000+00:...	9	12.05	9.100000000000001

10 rows | 0.20s runtime

Refreshed now

7. Destroy Infrastructure



```
azurerm_resource_group.bdcc: Destruction complete after 47s
random_string.suffix: Destroying... [id=xdpv]
random_string.suffix: Destruction complete after 0s
Releasing state lock. This may take a few moments...

Destroy complete! Resources: 5 destroyed.
askaniusz@askaniusz-Crosshair-16-HX-D14VFKG:~/Desktop/m07_sparksql_python_azure-master/terraform$
```